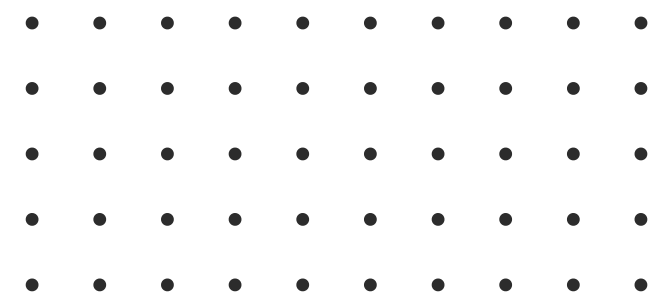


Toxic Detection for Text Messages

Sentiment Analysis



Thant Zin Myo - 6837929

Background

live streaming app

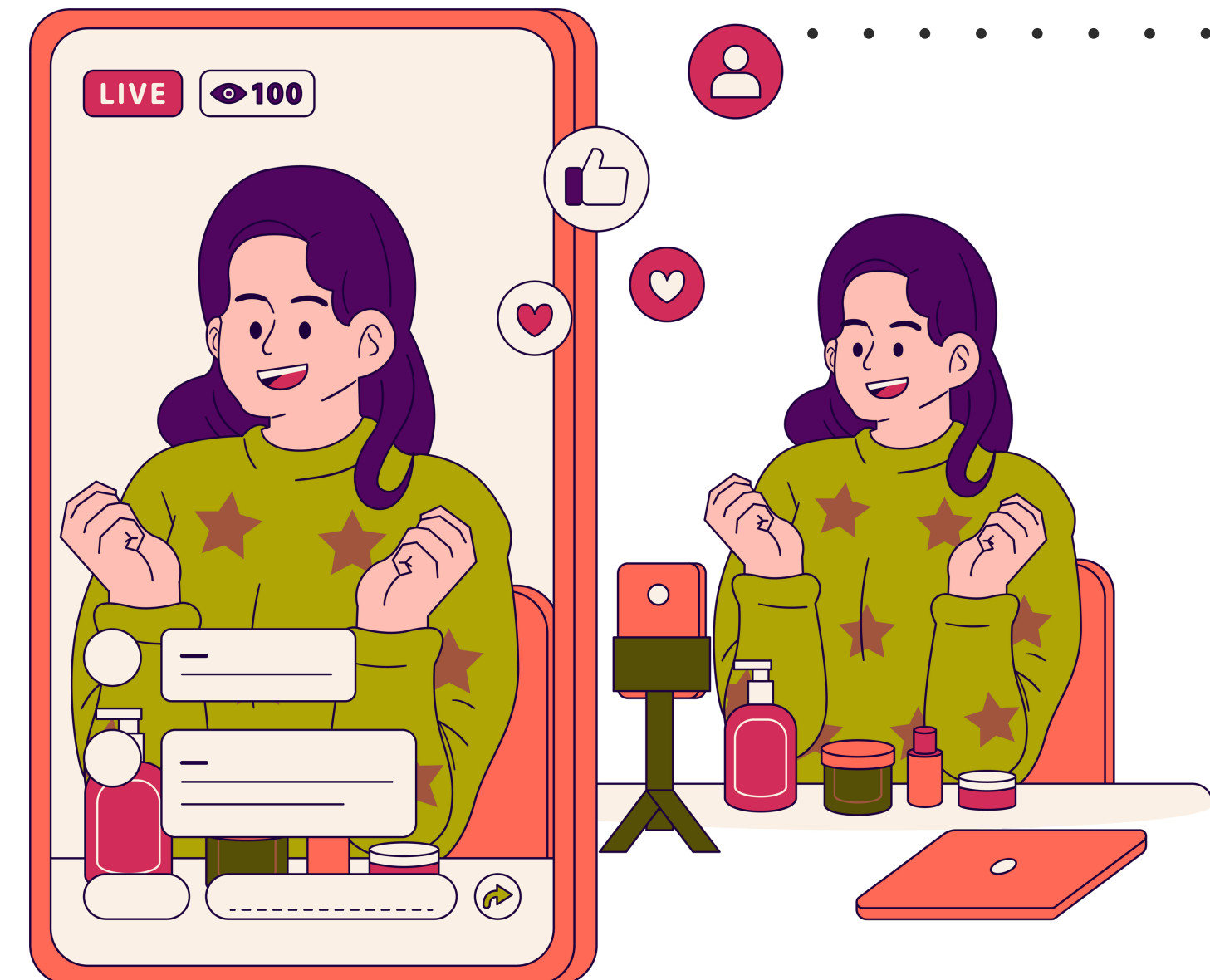
viewers can interact in real time by

- sending chat messages
- stickers

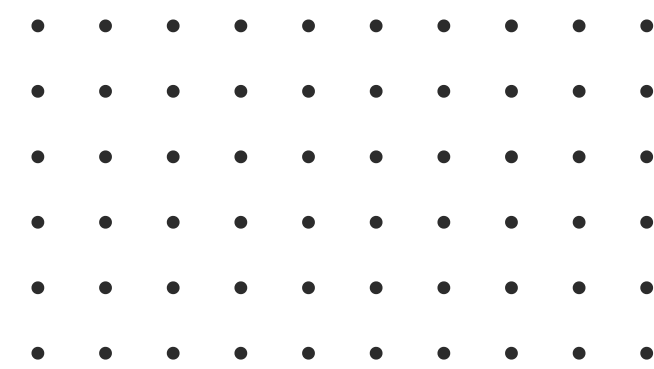
like Twitch or TikTok.

the chat often includes toxic or hateful

- harm the streamer
- create a negative environment



Data Source Selection



Source

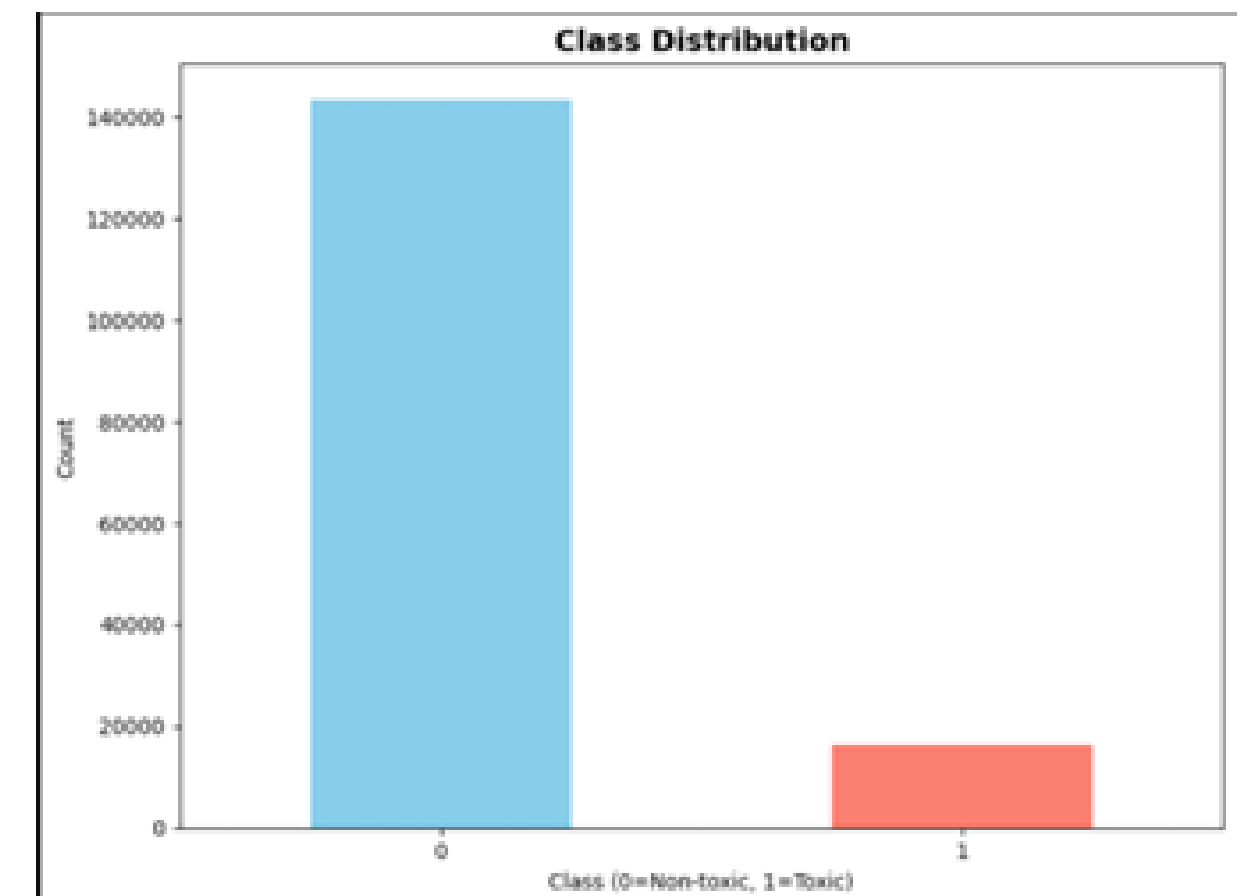
Kaggle. (2018). Jigsaw Toxic Comment Classification Challenge

```
EXPLORATORY DATA ANALYSIS
Label Distribution:
Non-toxic (0): 143,346 (89.8%)
Toxic (1): 16,225 (10.2%)
Average text length: 394 characters
Average word count: 67 words
```

Reason

- Real World Dataset
- learger dataset when compare to other online available dataset
- challenge myself with unbalance dataset

Note :
six toxic categories: **toxic, severe toxic, obscene, threat, insult, and identity hate.**
combined all of them into one label (**toxic/non-toxic**) as we will do **Sentiment Analysis**



Exploring Data

EXPLORATORY DATA ANALYSIS

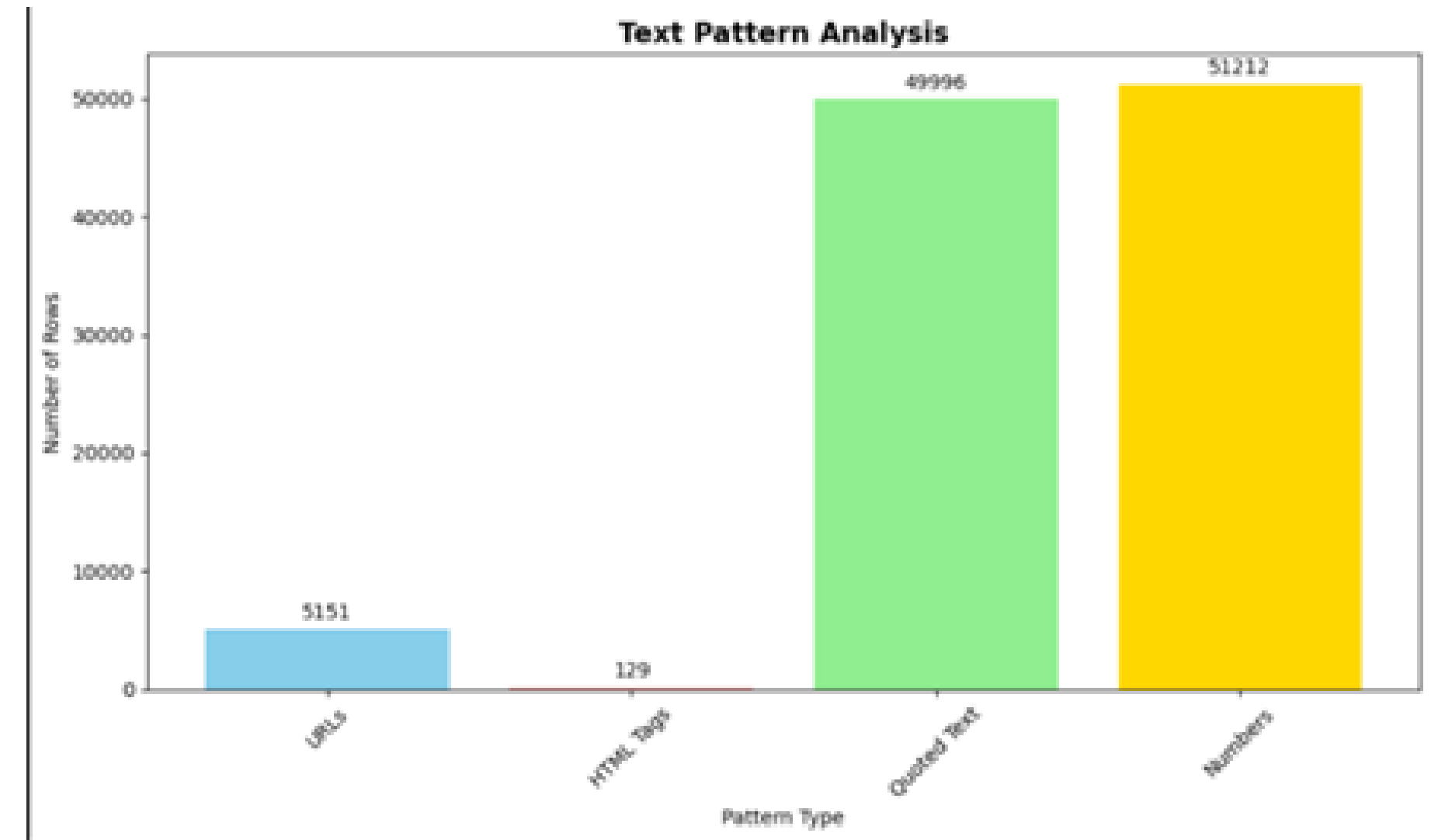
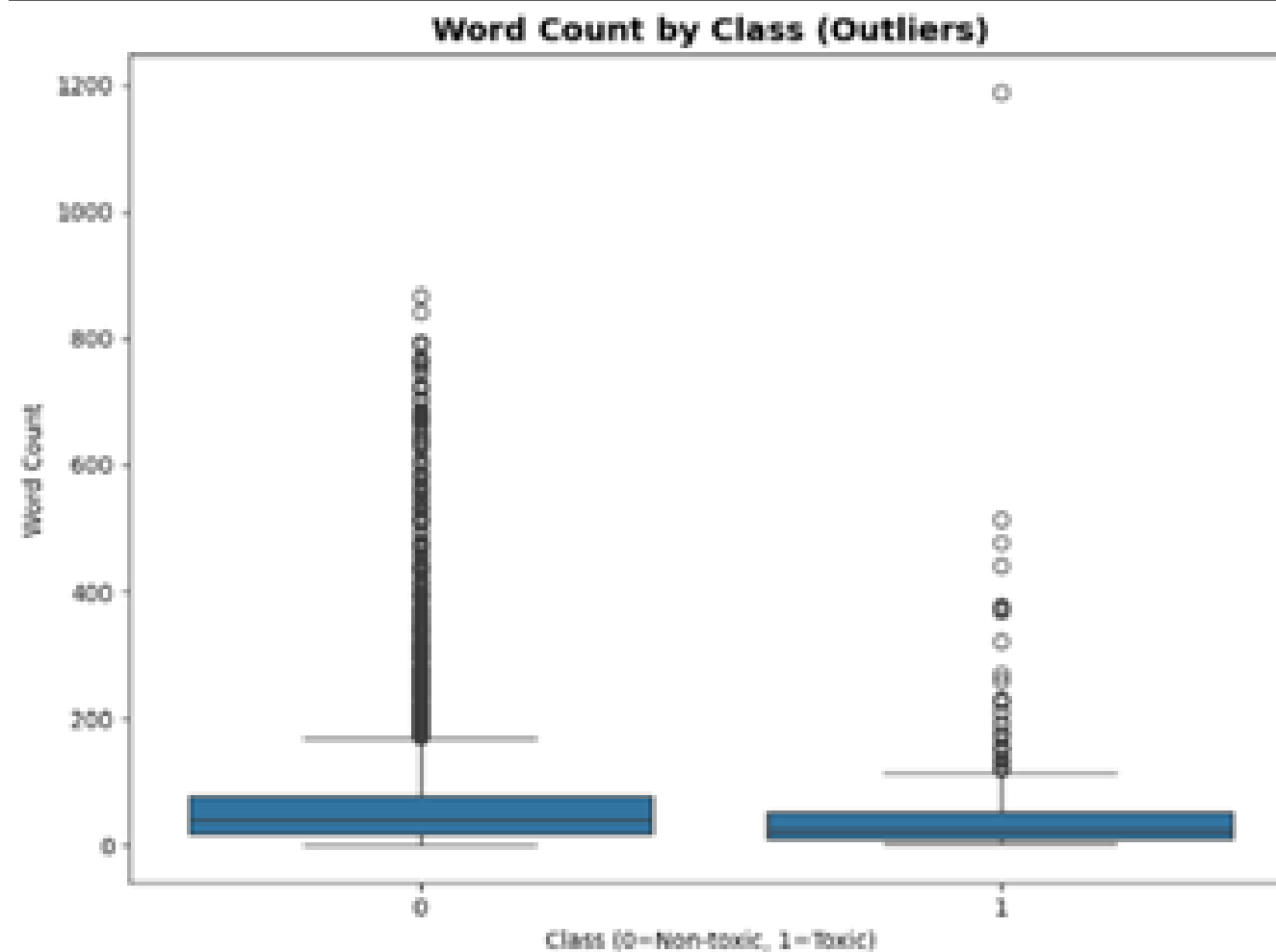
Label Distribution:

Non-toxic (0): 143,346 (89.8%)

Toxic (1): 16,225 (10.2%)

Average text length: 394 characters

Average word count: 67 words



Data Cleaning

Cleaning Steps

- Lowercasing text
- Removing numbers, quotes, URLs, and HTML tags
- Expanding contractions (e.g., don't -> do not) [link](#)
- Removing punctuation (keeping only alphabetic tokens)
- Stripping extra whitespace
- Tokenization & Lemmatization
 - Tokenization: Splitting text into words using NLTK.
 - Stop word removal: Common English words removed
 - (e.g., “the”, “and”).
 - Lemmatization: Words reduced to their base form
 - (e.g., running -> run).

```
# expand common contractions (small set)
url = "https://raw.githubusercontent.com/andrewburt/contractions/master/contractions.json"
response = requests.get(url)
data = response.json()
# Convert to simple dictionary (take first option from list)
contractions = {}
for key, value in data.items():
    contractions[key] = value[0]

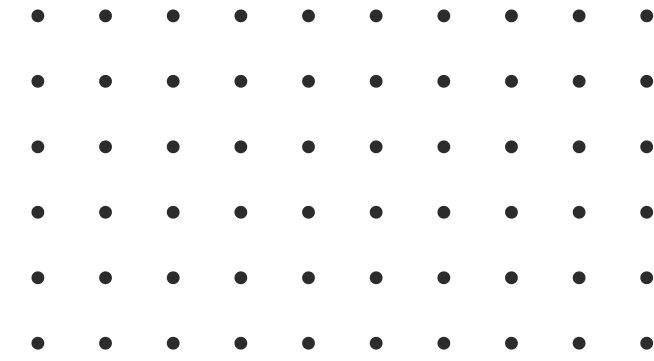
def basic_clean(text):
    if not isinstance(text, str):
        return ""
    text = text.lower()
    # Remove numbers
    text = re.sub(r'\d+', " ", text)
    # Remove quoted text
    text = re.sub(r'"[^"]*"', " ", text)
    # Remove URLs
    text = re.sub(r"http\S+|www\S+", " ", text)
    # Remove HTML tags
    text = re.sub(r"<.*?>", " ", text)

    for k, v in contractions.items():
        text = text.replace(k, v)
    # remove others except english word
    text = re.sub(r"^[^a-z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text
```

```
# Write lemmatization and tokenization
def tokenize_and_lemmatize(text, lemmatizer, stop_words):
    tokens = word_tokenize(text)
    # Remove stop_word
    tokens = [t for t in tokens if t not in stop_words and t.isalpha()]
    # lemmatize
    lem = [lemmatizer.lemmatize(t) for t in tokens]
    return " ".join(lem)
```

✓ 0.0s

Modeling Setting



1. Multinomial Naive Bayes

- Setting:
 - TF-IDF
 - Cleaning & remove stop word
 - Tokenization & Lemmatization

2. Multinomial Naive Bayes

- Setting:
 - Bag of words (feature =10000+ lexicon),
 - Cleaning & remove stop word
 - Tokenization & Lemmatization

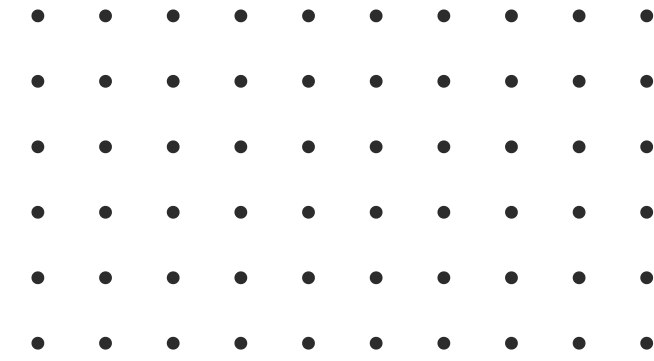
3. Logistic Regression

- Setting:
 - TF-IDF
 - Cleaning & remove stop word
 - Tokenization & Lemmatization

4. Logistic Regression

- Setting:
 - Bag of words (feature =10000+ lexicon),
 - Cleaning & remove stop word
 - Tokenization & Lemmatization

Modeling Setting



1. Multinomial Naive Bayes

- Setting:
 - TF-IDF
 - Cleaning & remove stop word
 - Tokenization & Lemmatization

2. Multinomial Naive Bayes

- Setting:
 - Bag of words (feature =10000+ lexicon),
 - Cleaning & remove stop word
 - Tokenization & Lemmatization

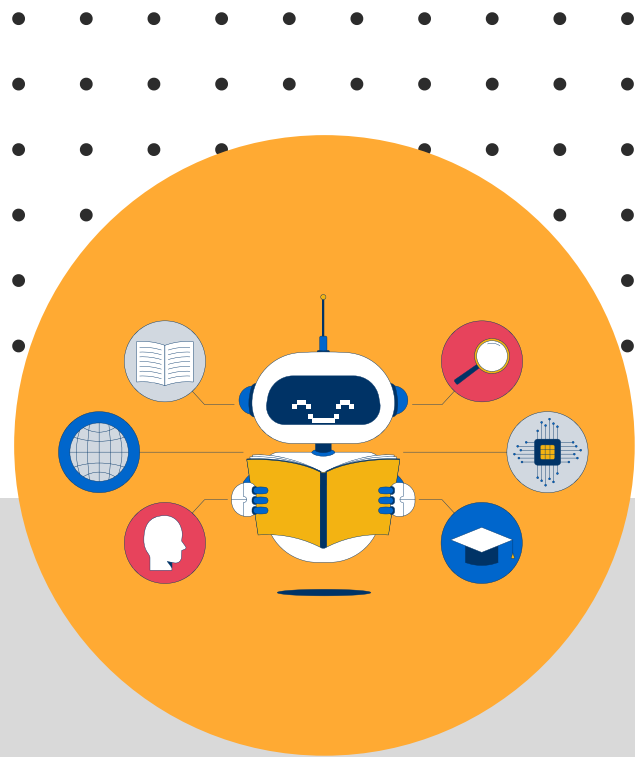
3. Logistic Regression

- Setting:
 - TF-IDF
 - Cleaning & remove stop word
 - Tokenization & Lemmatization

4. Logistic Regression

- Setting:
 - Bag of words (feature =10000+ lexicon),
 - Cleaning & remove stop word
 - Tokenization & Lemmatization

Training & Testing Dataset



without lexicons

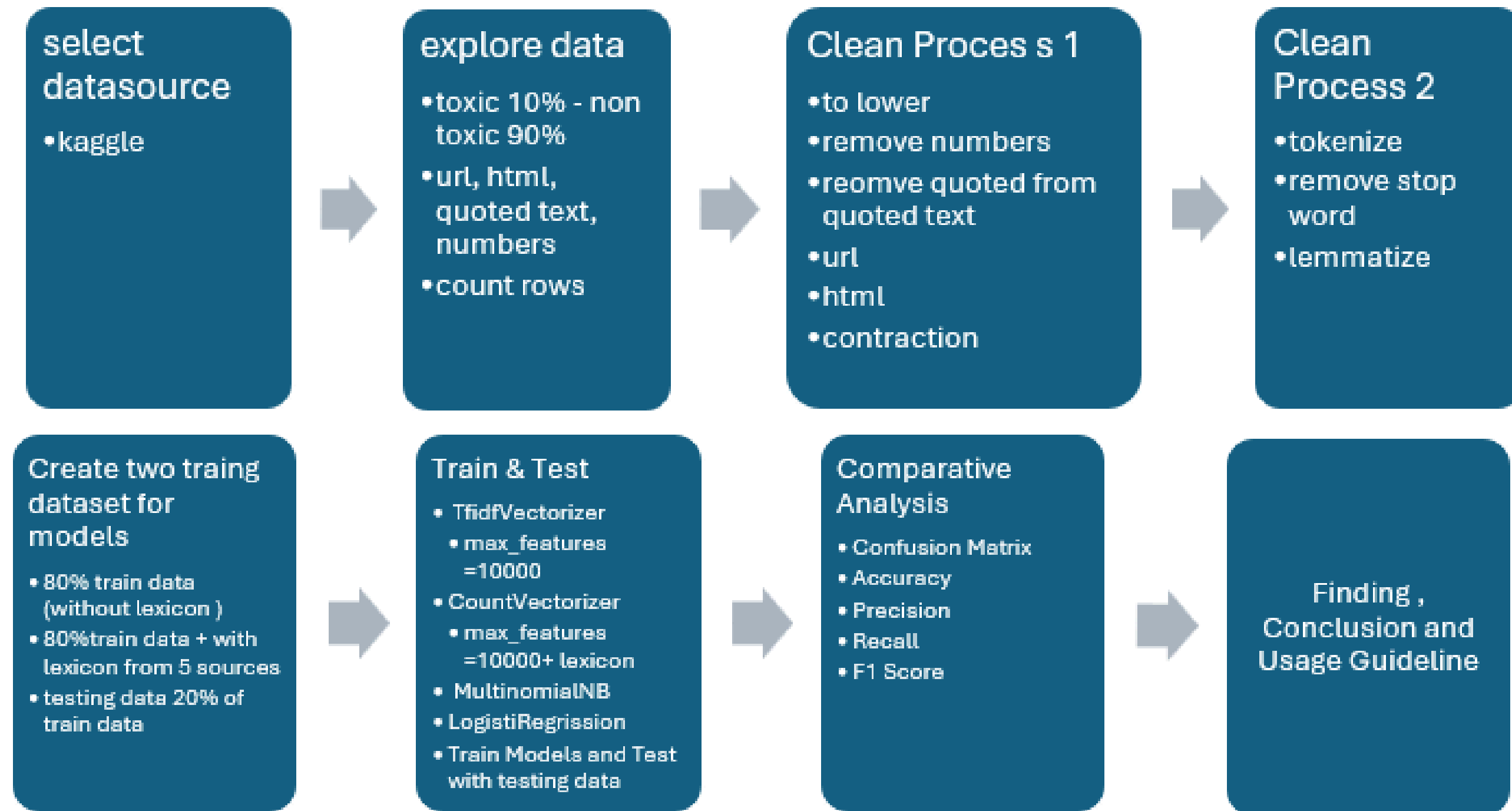
- 80% for training (127,656 samples)
- 20% for testing (31,915 samples)



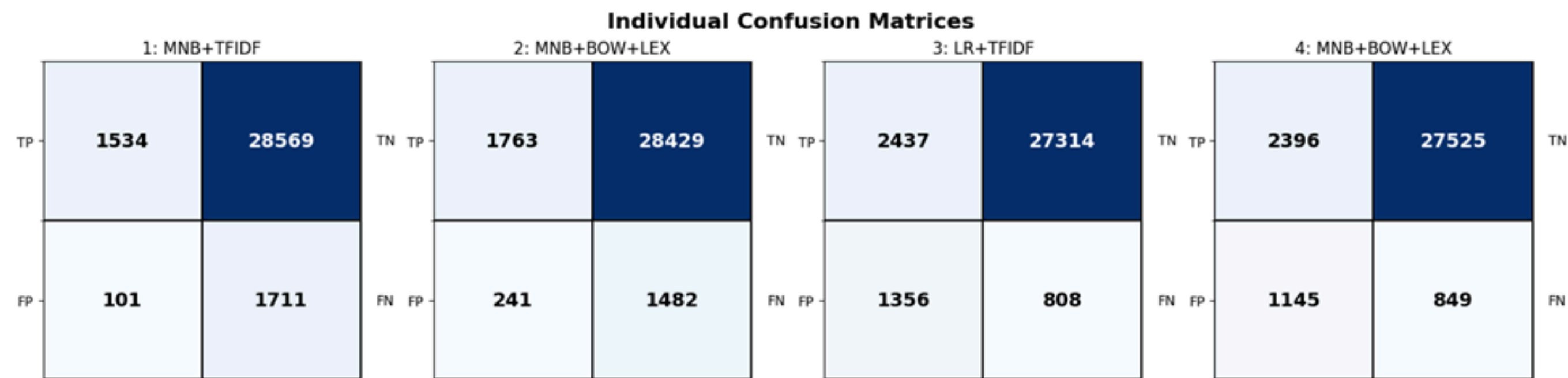
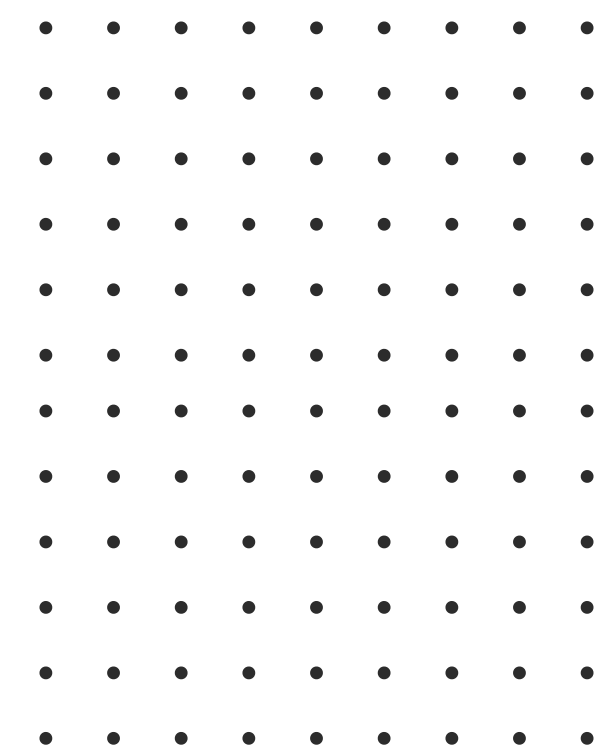
with lexicons

- 80% for training (127,656 samples)+ (lexicon word as text and label as 1) = 132, 416
- 20% for testing (31,915 samples)

Full Process



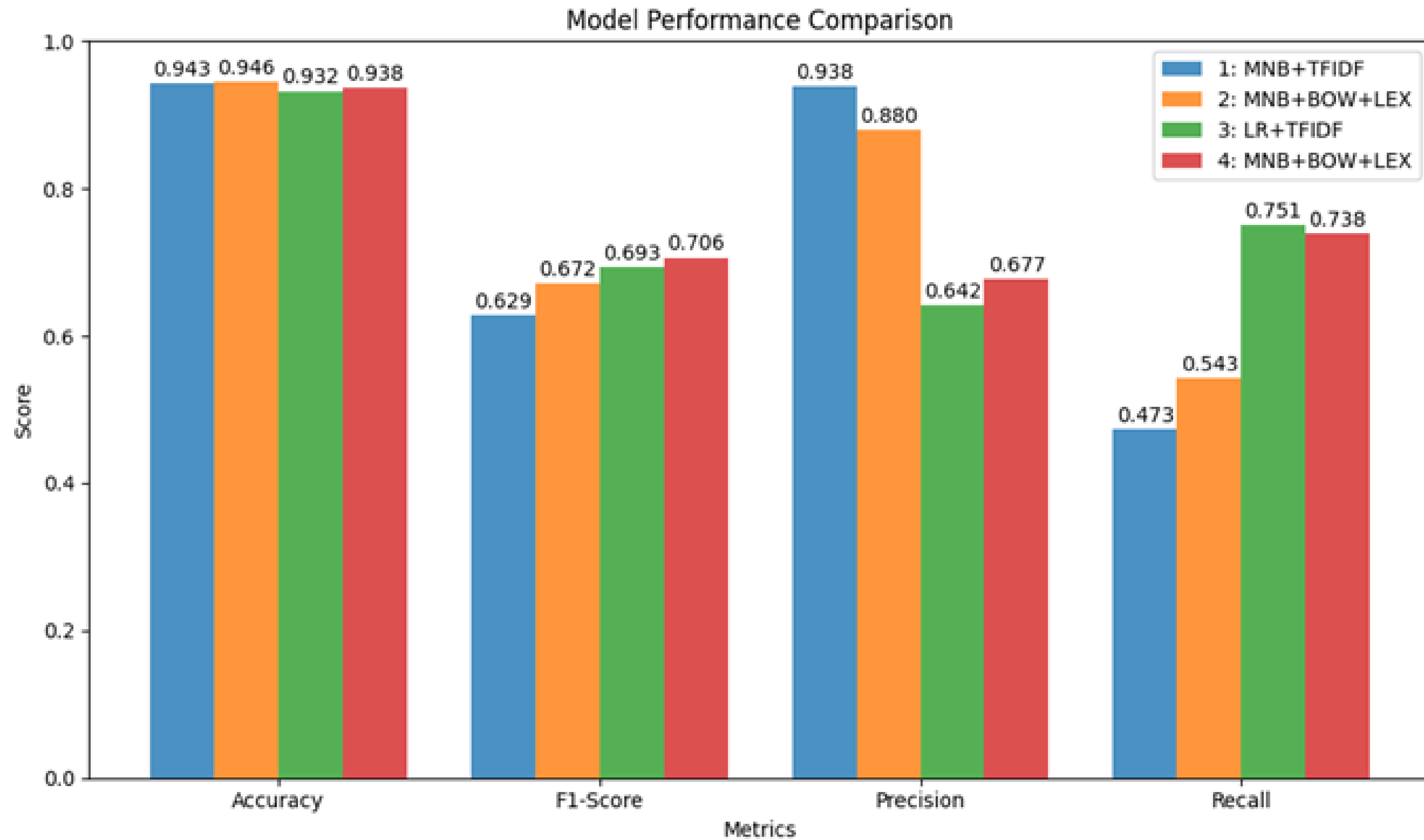
Result And analysis



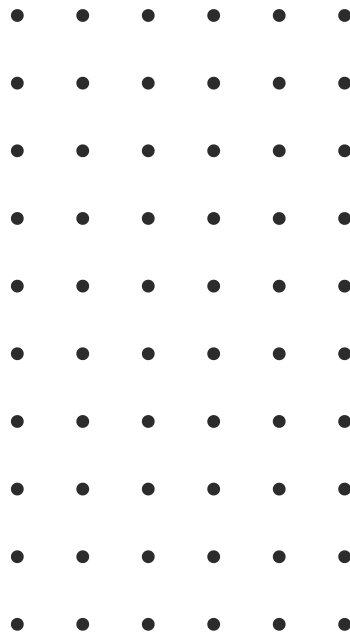
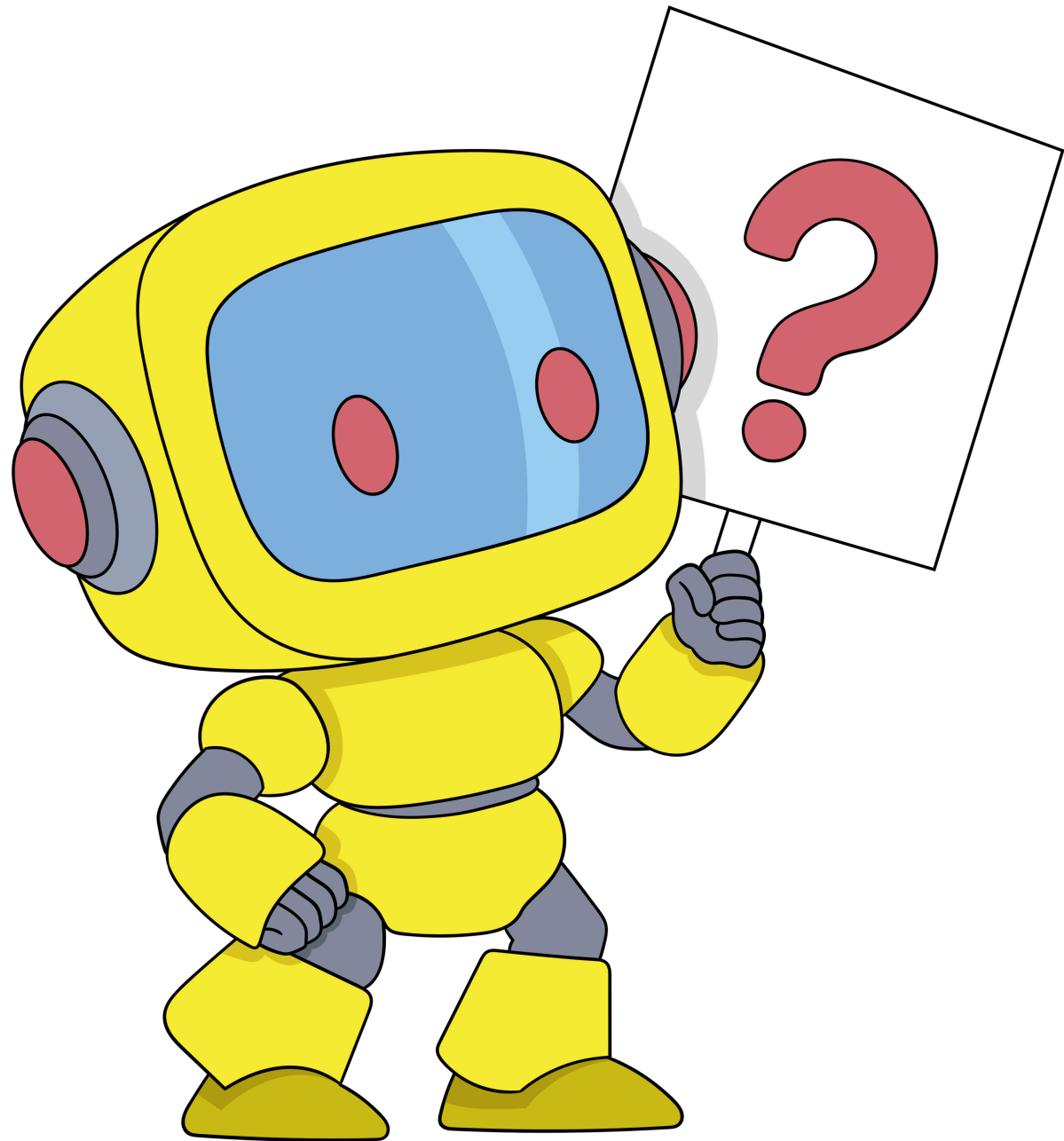
FINAL COMPARISON SUMMARY

Model	Accuracy	F1-Score	Precision	Recall	True Positives	True Negatives	False Positives	False Negatives
1: MNB+TFIDF	0.9432	0.6287	0.9382	0.4727	1534	28569	101	1711
2: MNB+BOW+LEX	0.946	0.6717	0.8797	0.5433	1763	28429	241	1482
3: LR+TFIDF	0.9322	0.6925	0.6425	0.751	2437	27314	1356	808
4: MNB+BOW+LEX	0.9375	0.7062	0.6766	0.7384	2396	27525	1145	849

F1 - Score



Question & Answer



During This Session
you can try my trained model performances with

<https://nlp-0vmq.onrender.com/>

which i run on render cloud